RAGIX

Sovereign document & code intelligence — digest, analyse, compare and generate large heterogeneous corpuses, entirely on your own infrastructure.

Sovereign | **Documents & code** | **Auditable**

AI-first engineering, Human-first results

June 2026 RAGIX v0.71 Adservio Innovation Lab

Contents

The challenge

Documents and code, growing faster than anyone can read them

Every large organisation accumulates two corpuses that outgrow human attention — and **the most valuable question sits exactly between them**.

Hundreds of documents

Specifications, architecture notes, audits and regulations, in mixed formats and languages, written over years by many hands.

- Mixed PDF, Office, Markdown
- Multiple languages
- Overlaps, gaps and drift go unnoticed

Tens of thousands of classes

Codebases whose real behaviour has quietly drifted from what the documentation claims.

- 10^3–10^5 classes
- Undocumented change over a decade
- Hard to audit by hand

Is the code actually doing what the specifications say — across thousands of files, and on material that cannot leave your premises?

RAGIX in one minute

A sovereign engine that turns large corpuses into trustworthy deliverables

RAGIX **reads, structures, summarises, compares, translates and generates** — entirely on your own infrastructure, with every step recorded.

Sovereign

Runs inside your perimeter; air-gap ready; no external API. Usable on regulated material.

Local-first

Private local models via Ollama. No vendor lock-in, predictable cost, works offline.

Frugal & exact

Kernels compute every metric; models only reason. The same input always yields the same numbers.

Auditable

Every command, edit and model call recorded in a tamper-evident SHA-256 chain.

What makes RAGIX unique

Five design choices that are hard to find together — especially on-premises

Differentiator	Why it matters
Facts & kernels generate documents	No hallucinated numbers; reproducible, defensible deliverables
Sovereign, with proof	Runs locally and can prove an external AI never read your content
Reasoning in layers	Handles corpuses far larger than any context window
Graph-RAG & projections	Retrieval guided by provenance — prevents false links across versions
End-to-end provenance	A tamper-evident SHA-256 chain — audit-ready evidence on demand

Facts and kernels generate the document

Not prompts — this is how RAGIX removes hallucination from the deliverable itself

In a RAGIX report, summary or slide deck, **every figure, table and metric is computed by deterministic kernels and tied to a source**. A language model is asked only to phrase the sentences around material that is already true — never to author the facts.

Prompt-only generation

Asking a model to "write the report" lets it produce text that reads convincingly but cannot be trusted — the classic failure mode of generative AI.

- Numbers can drift or be invented
- Statements are unverifiable
- Output is not reproducible

RAGIX: facts first, prose last

The document is assembled from verified facts and computed values; the model only phrases what is already proven and checked.

- Numbers are computed, never written
- Every claim is tied to a source
- Reproducible byte-for-byte

> **We do not prompt a model to write your report — we compute it,** then let the model phrase what is already verified.

How a RAGIX document is written

Facts first, prose last — built outward from verified material, not improvised from a prompt

Extract
Citeable facts pulled from the corpus, each with a source pointer

Compute
Tables, figures and metrics calculated by kernels

Assemble
Document skeleton, sections and references built

Write
Model phrases prose around facts already verified

Verify & lock
A tutor model checks; tables, code and maths untouched

Documents RAGIX generates this way — produced on real engagements

Generated artefact	What is machine-derived, not written
Code audit report (FR / EN)	Every metric, hotspot, coupling table and risk score
Multi-level corpus summary	Fact catalogue, citations, cross-references and appendices
Executive slide deck	Slide structure, figures and quoted findings from the report
Glossary & handouts	Acronyms and definitions extracted from the source corpus

Re-run it and you get the same numbers and tables, byte for byte — an AI-written document that stands as evidence.

The capability catalogue

Six kernel families — six business capabilities, one engine

Translation and multilingual restitution are not a separate family but a property of the engine — French and English are produced routinely, with the source meaning preserved.

Family	It lets you…	On material such as…
Audit	Analyse code quality, complexity, coupling and risk	Java / JEE codebases of 10^3–10^5 classes
Docs	Digest & summarise hierarchically, with citations	Hundreds of PDFs, Office and Markdown files
Summary	Compare & consolidate across sources and versions	Multi-vendor, multi-release document sets
Reviewer	Review & edit long documents, fully traceable	50+ page specifications and reports
Presenter	Generate slide decks from large reports	Long reports → 25–120 slide decks
Security	Scan for vulnerabilities and compliance gaps	Dependencies, secrets, configuration

"One engine: digest, analyse, compare, translate, generate"

Reasoning in layers, retrieval by provenance

How RAGIX captures meaning across corpuses larger than any context window

Layered reasoning

A corpus never fits in one prompt, so RAGIX reasons in disciplined, checkable steps.

- Classify the problem
- Decompose into sub-problems
- Solve each independently
- Verify and reconcile results

Three retrieval modes

Retrieval is chosen to fit the task — and guided by provenance, not similarity alone.

- **Unix-RAG** — live, millisecond code exploration
- **Project-RAG** — hybrid keyword + semantic search
- **Graph-RAG** — provenance-constrained synthesis that keeps versions and sources distinct

Orchestratable by an AI — sovereignly

An external assistant can drive the work without reading a single document

Requests travel inward through a guarded gateway; **only sanitised results travel back out**. A top-tier assistant such as Claude can orchestrate the analysis while content stays sealed in your perimeter.

Level	For	Contains	Excludes
Internal	The audit team	Full traces, hashes, paths	Nothing
External	Client delivery	Clean report, metrics, findings	Paths, IDs, metadata
Orchestrator	An external AI	Metrics only: counts, timings	All document text
Compliance	A regulator	Hash chain + attestations	Content (trail only)

Proof, not promise: every model call was local and every boundary crossing carried metrics only — with a cryptographic chain to show it.

Proof at scale

Field engagements on regulated material, entirely on-premises

Lines of code audited

88K

Across enterprise Java / JEE platforms

Documents analysed

159

PDF, Office and slides, 5,515 indexed segments

Full code audit run

3.4s

A 60K-line codebase, three-stage pipeline

Classification accuracy

97%

Validated by client product owners

Weeks of analysis compressed into seconds and hours — every figure computed, attested and reproducible.

Field case studies

Anonymised, sectoral — figures drawn from delivered work

Enterprise Java platform under maintenance

Rapid, objective health assessment for cost and risk planning.

- 806 files · 60,157 lines · 582 classes
- Full audit in 3.4 seconds
- Overall grade A; hotspots pinpointed

National gas-distribution operator

Feasibility of separating two entangled business domains.

- ~88,000 lines · 952 classes · 5,834 dependencies
- Cross-domain coupling 0.36% — separation viable
- 97% auto-classified; costed roadmap delivered

Major public administration

Sprawling document corpus analysed under a strict on-premises requirement.

- 159 documents · 5,515 segments · 9 themes
- 109 capabilities catalogued; 51 discrepancies flagged
- 100% local — zero external calls

Deliverables were produced in both French and English — reports, executive decks, glossaries and handouts.

Operating an advanced engineering forge

RAGIX as the sovereign intelligence layer over engineering at scale

A modern software forge holds code, builds, documentation and collaboration together. At scale the hard problem becomes **understanding** — keeping code, specifications and documentation provably in step.

Governance

Every analysis reproducible and attested, with tiered outputs for teams, clients and regulators.

Scale

10^3–10^5 classes and hundreds of documents handled in seconds to hours.

Sovereignty

The intelligence layer runs inside the same trusted perimeter as the forge itself.

Velocity

Audits, consistency checks, summaries and restitutions produced on demand.

Mature, open, and ready

Production-ready, open source, built and operated by Adservio Innovation Lab

Why RAGIX

- **Sovereign & auditable** — runs on your infrastructure
- **Broad & coherent** — one engine for documents and code
- **Proven & fast** — field-tested at 88K lines, 159 documents
- **Open & owned** — open source, no vendor lock-in

Find out more

- v0.71 · production-ready · local-first stack
- Open source — github.com/ovitrac/RAGIX
- Demo: full audit on YouTube — youtu.be/vDHI70ZPnDE
- Olivier Vitrac, PhD, HDR — olivier.vitrac@adservio.fr
- Adservio Innovation Lab — Tour Franklin, 100-101 Terrasse Boieldieu, 92800 Puteaux — La Défense